

CSA43 - Assessment Tool 2 Answers

COURSE NAME: Internet Programming **COURSE CODE:** CSA43

COURSE OUTCOME COVERED

CO-3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool: Technical Presentation **Weightage:** 20% **Total Marks:** 20

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: _____

1. Servlet Architecture & Workflow (4 Marks)

Question: Present the architecture of Java Servlets and explain the complete request-response cycle with a neat diagram.

Client-Server Interaction

The browser (client) sends an **HTTP request** (a GET or POST) to a web server URL that maps to a servlet. The client has no knowledge of how the request is processed internally — it only understands HTTP request/response semantics. The server processes the request and returns an **HTTP response** (status line, headers, and body such as HTML/JSON), which the browser renders.

Role of the Web Container

The **Web Container** (e.g., Apache Tomcat, Jetty) is the runtime environment that actually executes servlets. It is responsible for: - Mapping incoming URLs to the correct servlet (via `web.xml` or `@WebServlet` annotations). - Instantiating the servlet (once) and managing its life cycle (`init`, `service`, `destroy`). - Creating a fresh `HttpServletRequest` and `HttpServletResponse` object for every request and passing them to `service()`. - Multithreading: spawning a separate thread per request so multiple clients can be served concurrently by one servlet instance. - Providing supporting services: session management, security, JNDI resource lookup.

Request and Response Objects

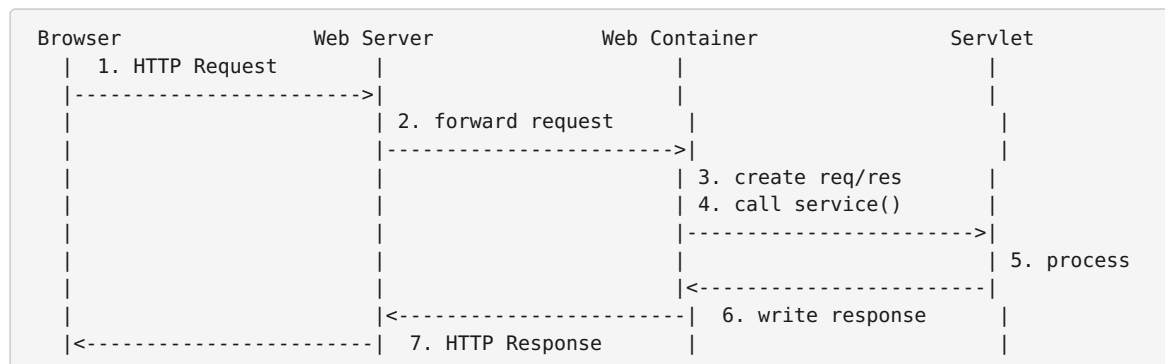
- **HttpServletRequest** - encapsulates everything about the incoming request: form parameters (`getParameter()`), headers, cookies, the session, and the request method.
- **HttpServletResponse** - used by the servlet to build the outgoing response: set the content type, status code, headers, cookies, and write the response body through a `PrintWriter/OutputStream`.

Request-Response Cycle (Step by Step)

1. User submits a request (e.g., clicks a link or submits a form) → browser creates an HTTP request.
2. Request reaches the **Web Server**, which forwards it to the **Web Container**.
3. Container consults the deployment descriptor / annotations to find the matching servlet.
4. If this is the servlet's first invocation, the container instantiates it and calls `init()`.
5. Container creates `HttpServletRequest` and `HttpServletResponse` objects and invokes `service()` on a new thread.
6. `service()` dispatches to `doGet()` or `doPost()` based on the HTTP method.

7. The servlet executes business logic — reads parameters, talks to a database via JDBC if needed.
8. The servlet writes output into the response object.
9. The container sends the completed HTTP response back through the web server to the browser.
10. The browser renders the response (HTML page, JSON, redirect, etc.).

Diagram (conceptual, to redraw on a slide):



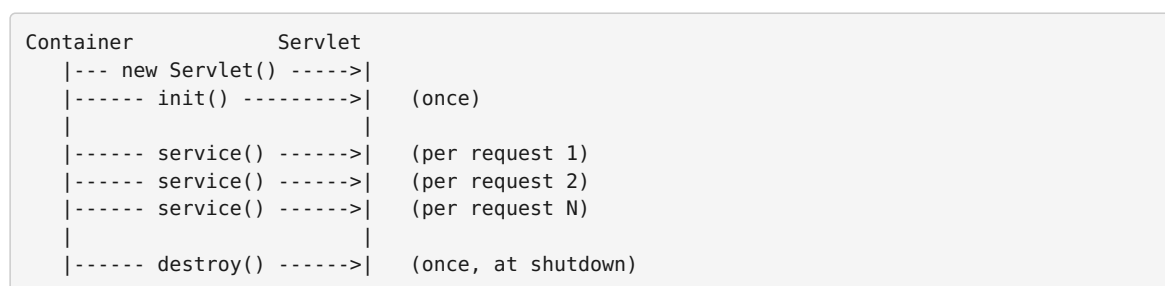
2. Servlet Life Cycle Analysis (4 Marks)

Question: Explain the Servlet Life Cycle with a sequence diagram and analyze the role of `init()`, `service()`, and `destroy()` methods.

Life Cycle Phases

1. **Loading and Instantiation** - The container loads the servlet class and creates a single instance (either on the first request, or eagerly at startup if `load-on-startup` is configured).
2. **Initialization** - `init()` - Called exactly once, immediately after instantiation, before the servlet handles any request. Used for one-time setup: reading `ServletConfig` init-parameters, opening a connection pool, loading static/reference data.
3. **Request Handling** - `service()` - Called once per incoming request, on its own thread. `HttpServlet.service()` internally examines the HTTP method and dispatches to `doGet()`, `doPost()`, `doPut()`, `doDelete()`, etc.
4. **Destruction** - `destroy()` - Called exactly once, when the container decides to take the servlet out of service (server shutdown, application redeploy, or explicit unload). Used to release resources cleanly: close connections, save in-memory state, flush logs.

Sequence Diagram (conceptual)



Execution Flow

Constructor → `init()` → [`service()` → `doGet()/doPost()`] (repeated for every request, concurrently on multiple threads) → `destroy()`

Performance Implications

- Because `init()` runs only once, expensive setup (DB connection pool, loading configuration) done there is **amortized across all requests**, giving much better performance than opening

resources on every call.

- Because the container reuses a **single servlet instance** across many threads, servlets are lightweight and fast compared to spawning a new process/object per request (as older CGI scripts did).
- The multithreaded `service()` model means the servlet must be written **thread-safe**: avoid mutable instance/static variables that hold per-request state, since concurrent requests share the same object.
- `destroy()` gives a controlled shutdown window to release resources gracefully; skipping proper cleanup here can cause memory/connection leaks across redeployments.

3. GET vs POST Method Evaluation (4 Marks)

Question: Compare GET and POST methods used in form submission. Present scenarios where each method is appropriate.

Data Transmission

- **GET** appends form data to the URL as a query string (`?name=value&...`), sent as part of the request line.
- **POST** places form data inside the HTTP request body, separate from the URL.

Security Aspects

- GET data is visible in the browser's address bar, browser history, bookmarks, and server access logs — unsuitable for passwords or personal data.
- POST data does not appear in the URL, offering better (though not absolute — HTTPS is still required) protection for sensitive form submissions such as login credentials or registration details.
- GET requests can be cached and re-triggered by simply revisiting a URL, which is risky if the action changes server state; POST requests are not cached or repeated by the browser automatically, reducing accidental duplicate operations.

URL Limitations

- GET is constrained by maximum URL length (practically limited by browsers/servers, often ~2000 characters), making it unsuitable for large payloads or file uploads.
- POST has no such practical size restriction and is the only viable method for submitting files or large amounts of form data.

Scenario-Based Comparison

Scenario	Recommended Method	Reason
Search box on a website	GET	Safe, idempotent, shareable/bookmarkable URL (<code>?q=servlet</code>)
Pagination (<code>?page=2</code>)	GET	Read-only, benefits from being bookmarkable
Login form	POST	Sensitive credentials must not appear in the URL
Student/Product registration form	POST	Sensitive/large data, changes server state
File upload	POST	GET cannot carry binary/large payloads
Filtering a product catalog	GET	Non-sensitive, read-only, good for shareable links

Conclusion: GET is appropriate for safe, idempotent, read-only operations where shareable/bookmarkable URLs are useful; POST is appropriate whenever the operation changes

server state or the data is sensitive or large.

4. Session Tracking Techniques (4 Marks)

Question: Present different session tracking mechanisms (HttpSession, Cookies, URL rewriting). Evaluate their advantages and limitations.

HttpSession

Server-side object created via `request.getSession()`, identified to the client through a session ID (commonly transported via a `JSESSIONID` cookie). Stores arbitrary attributes on the server for the duration of the user's visit.

- **Advantages:** Highly secure (actual data never leaves the server), can hold complex Java objects, simple API (`setAttribute/getAttribute`).
- **Limitations:** Consumes server memory per active user; needs session replication/sticky sessions to scale across a server cluster; relies on cookies or URL rewriting under the hood to carry the session ID.

Cookies

Small key-value pairs ($\leq \sim 4$ KB) that the server asks the browser to store (`Set-Cookie` header) and which the browser automatically resends on every subsequent request to the same domain.

- **Advantages:** Stateless on the server (no memory overhead per user), can persist across browser restarts (`setMaxAge`), simple to use for non-sensitive preferences.
- **Limitations:** Limited storage size and count per domain; visible/editable on the client, so unsuitable for sensitive data unless encrypted/signed; users can disable cookies entirely.

URL Rewriting

The session ID is manually appended to every hyperlink/form action as a URL parameter (e.g., `page.jsp;jsessionid=ABC123`), used as a fallback when the client has cookies disabled.

- **Advantages:** Works even when cookies are completely disabled in the browser.
- **Limitations:** Exposes the session ID directly in the URL (risk of leakage via browser history, referrer headers, shared links); every single link/form in the application must be manually rewritten (`response.encodeURL()`); less convenient for developers.

Use-Case Comparison

Mechanism	Best suited for
HttpSession	Login state, shopping cart contents, multi-step wizards/forms
Cookies	"Remember me" tokens, UI preferences, low-sensitivity tracking
URL Rewriting	Fallback session tracking when cookies are disabled by the client

Evaluation: In practice, `HttpSession` combined with a cookie (`JSESSIONID`) is the default and most secure approach; URL rewriting exists purely as a graceful-degradation fallback, and plain cookies alone are best reserved for non-sensitive data since they are fully visible and editable on the client side.

5. Cookies vs Sessions - Critical Analysis (4 Marks)

Question: Analyze and compare Cookies and Sessions in web applications. Present real-time use

cases.

Storage Location

- **Cookies** are stored entirely on the **client** (browser).
- **Sessions** (`HttpSession`) store the actual data on the **server**; only a small identifier (session ID) is exchanged with the client, typically via a cookie.

Security Risks

- Cookies can be read or modified by anyone with access to the browser/machine, and are vulnerable to interception if not sent over HTTPS or flagged `Secure/HttpOnly`; they can also be targeted by Cross-Site Scripting (XSS) if `HttpOnly` is not set.
- Sessions are inherently safer since sensitive data never leaves the server; the main residual risk is **session hijacking** if the session ID itself (carried in a cookie) is stolen — mitigated with HTTPS, `HttpOnly/Secure` cookie flags, and session timeouts.

Performance Considerations

- Cookies impose **no server memory cost**; the server stays fully stateless with respect to that data, which scales well horizontally across many servers.
- Sessions consume **server-side memory per active user**, and in a load-balanced/clustered deployment require session replication or a shared session store (e.g., Redis), adding operational complexity and slightly higher latency.

Side-by-Side Summary

Aspect	Cookies	Sessions (<code>HttpSession</code>)
Data location	Client (browser)	Server (memory/DB)
Capacity	Small (~4 KB), limited count	Large, can hold complex objects
Security	Lower – visible/editable client-side	Higher – data stays server-side
Lifetime control	Client-configurable expiry (<code>setMaxAge</code>)	Server-configurable timeout (<code>setMaxInactiveInterval</code>)
Scalability	Excellent (stateless server)	Requires replication for clustering
Survives browser restart	Yes, if persistent cookie	Typically no (unless client resends a still-valid session cookie)

Real-Time Use Cases

- **Cookies:** remembering a user’s preferred language/theme, a “Remember my username” box on a login page, storing a non-sensitive analytics/tracking ID, keeping items in a *guest* shopping cart before login.
- **Sessions:** maintaining a logged-in user’s authentication state across an online banking portal, storing multi-page form-wizard data (e.g., a multi-step registration or checkout flow), holding a shopping cart for a logged-in e-commerce user, storing role/permission information used for access control on every page.

Critical takeaway: Cookies and sessions are not competitors but complementary tools — a well-designed application uses a cookie purely to *identify* the client, while the actual sensitive application state lives safely in a server-side session, giving the best combination of security, scalability, and continuity.